

Chapter 3

User Registration



Tracing a registration request from the client through every backend layer — enforcing unique emails and securely hashing passwords with BCrypt.

3.1 Introduction

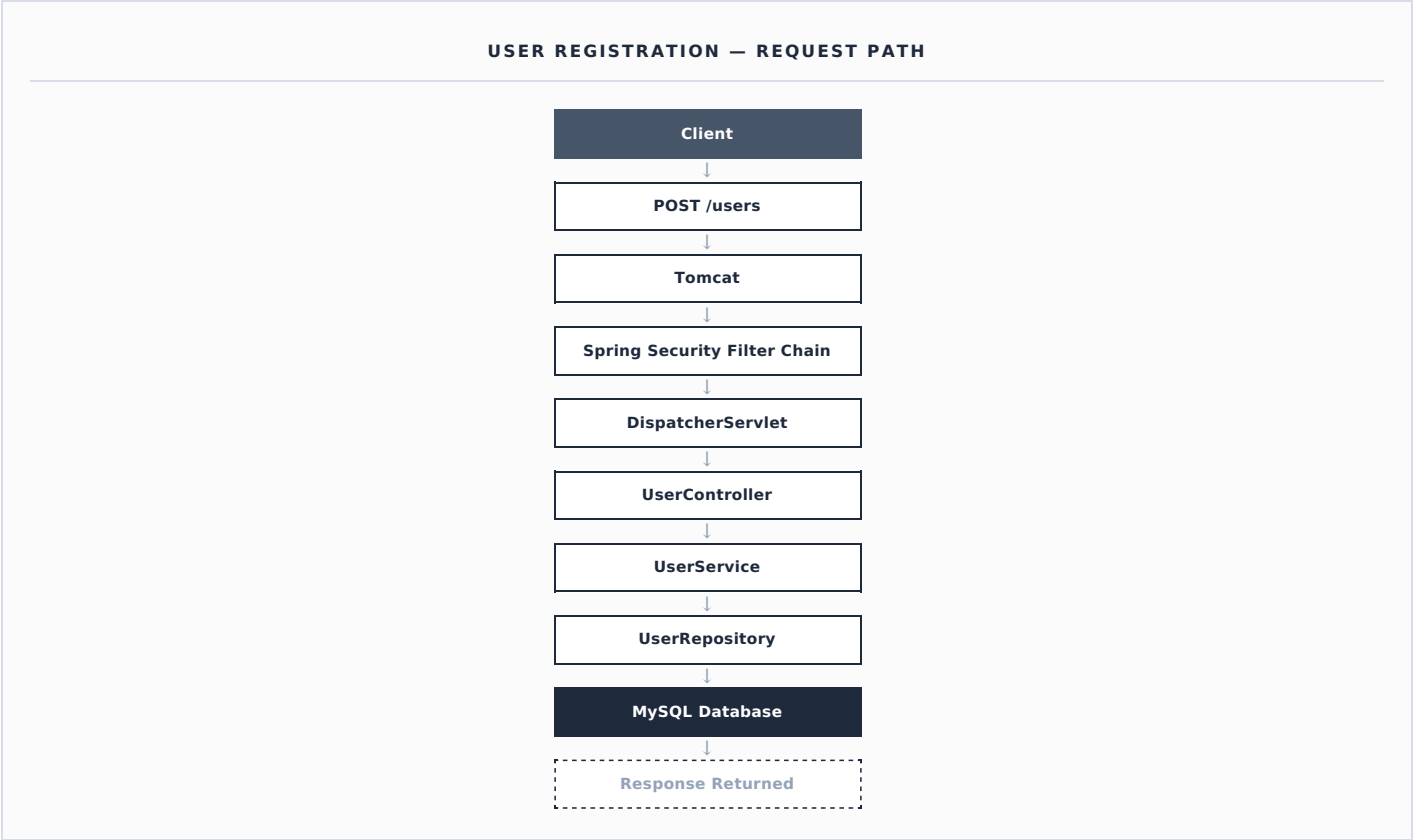
Now that our Spring Boot application has started successfully, the next step is to allow users to create an account.

Registration is the first interaction a user has with our backend. A user provides information such as their name, email, password, and role. Our responsibility is to validate the data, securely store it in the database, and return an appropriate response.

At first glance, registration appears to be a simple “save user” operation. However, several important tasks happen before the data reaches the database. We must ensure that duplicate accounts are not created, passwords are never stored in plain text, and each layer of the application performs only its own responsibility.

3.2 Registration Flow

The complete request flow for user registration is shown below. Although it looks like a simple sequence, each component performs a different task.



The Ten-Step Registration Sequence

1 Client Sends Registration Request

WHAT HAPPENS

The client (React application, mobile app, or Postman during development) sends an HTTP **POST** request to the registration endpoint:

```
{
  "name": "Rahul",
  "email": "rahul@example.com",
  "password": "password123",
  "role": "DONOR"
}
```

The request body contains all the information required to create a new user.

WHY POST?

HTTP provides different methods for different operations — GET reads data, POST creates new data, PUT updates existing data, and DELETE removes data. Since registration creates a new user, POST is the correct method.

2 Tomcat Receives the Request

WHAT HAPPENS

The request first reaches the embedded Tomcat server, which creates two important objects — `HttpServletRequest` and `HttpServletResponse` — that travel through the entire request lifecycle.

WHY

Spring Boot itself is not a web server. Tomcat acts as the web server that accepts incoming HTTP requests and forwards them into the Spring framework.

3 Spring Security Filter Chain

WHAT HAPPENS

Every request passes through the Spring Security Filter Chain before reaching the controller. For registration, the request usually does **not** contain a JWT because the user does not have an account yet. Our custom `JwtFilter` checks for an `Authorization` header; since no Bearer token is present, it simply calls the next filter in the chain.

WHY

Spring Security checks every request for consistency. Even though registration is a public endpoint, it still passes through the filter chain — the security configuration simply allows this endpoint without authentication.

4 DispatcherServlet

WHAT HAPPENS

After security processing, the request reaches the `DispatcherServlet` — the front controller of Spring MVC. Because the request URL matches `/users`, Spring routes the request to the registration method inside `UserController`.

WHY

Instead of every controller listening for requests independently, Spring uses a single entry point (`DispatcherServlet`) to route requests efficiently.

5 UserController

WHAT HAPPENS

The controller receives the request body and converts the incoming JSON into a `User` object:

```
@PostMapping
public User register(@RequestBody User user) {
    return userService.registerUser(user);
}
```

The controller does **not** contain business logic — it simply forwards the request to the service layer.

WHY

Controllers should focus on HTTP communication. Keeping business logic out of the controller makes the application easier to maintain and test.

6 UserService

WHAT HAPPENS

The service contains the business rules for registration, which follows these steps:

- Check whether the email already exists
- Encrypt the password
- Save the user
- Return the saved user

This is where the actual decision-making happens.

WHY

Business logic belongs in the service layer because multiple controllers or future APIs may need to reuse the same logic.

7 Check if Email Already Exists

WHAT HAPPENS

```
userRepository.existsByEmail(user.getEmail());
```

If the email already exists, an exception is thrown and registration stops.

WHY

Every user account should have a unique email address. Allowing duplicate emails would make login and account identification unreliable.

8 Encrypt the Password

WHAT HAPPENS

Before saving the user, the raw password is encrypted:

```
user.setPassword(passwordEncoder.encode(user.getPassword()));
```

The plaintext password is replaced with its BCrypt hash.

WHY

Passwords must never be stored in plain text. If the database is compromised, attackers should not be able to read users' passwords. BCrypt is designed specifically for password hashing and includes a random salt, making identical passwords produce different hashes.

9 Save the User

WHAT HAPPENS

After validation and password hashing, the service saves the user:

```
userRepository.save(user);
```

The repository communicates with Hibernate, which generates the appropriate SQL statement and stores the data in MySQL.

WHY

The repository layer isolates database operations from business logic. The service does not need to know SQL syntax or database-specific details.

10 Return the Response

WHAT HAPPENS

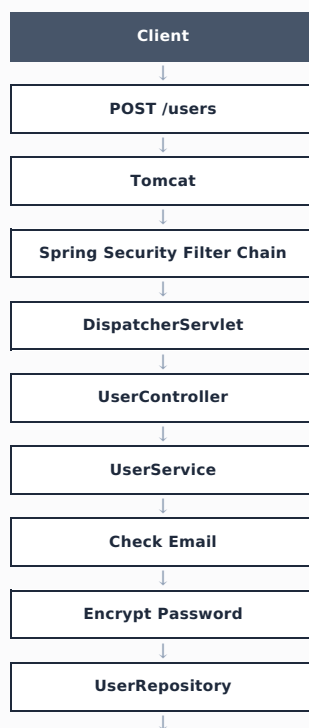
Once the user has been saved, the repository returns the saved entity to the service. The service returns it to the controller, and the controller sends an HTTP response back to the client. The registration process is now complete.

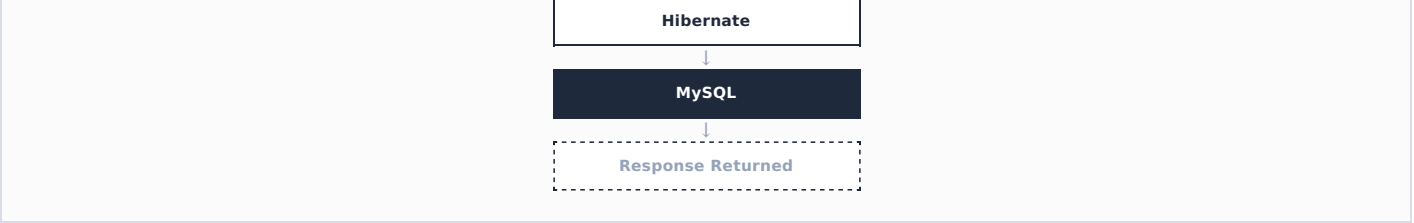
WHY

Each layer returns the result to the previous layer, following the same path the request took in reverse — keeping the request lifecycle structured and predictable.

Registration Flow Summary

FULL PATH — CLIENT TO DATABASE AND BACK





Important Concepts Learned

HTTP POST	@PostMapping	@RequestBody	Controller Layer	Service Layer	Repository Layer	existsByEmail()	PasswordEncoder
BCryptPasswordEncoder	Password Hashing	Spring Data JPA	Hibernate	MySQL			

COMMON MISTAKES

- **Storing passwords without hashing** — this is a serious security vulnerability.
- **Putting business logic inside the controller** — controllers should delegate work to services.
- **Skipping the duplicate email check** — this can lead to multiple accounts with the same email.
- **Calling the repository directly from the controller** — this breaks the layered architecture and makes the application harder to maintain.

CHAPTER SUMMARY

In this chapter, we implemented the user registration feature. We traced the complete journey of a registration request — from the client, through Tomcat, Spring Security, the DispatcherServlet, controller, service, repository, Hibernate, and finally the MySQL database.

Along the way, we enforced two essential business rules: preventing duplicate email addresses and securely hashing passwords with BCrypt before storing them. These practices form the foundation of a secure authentication system and prepare us for the next step: allowing registered users to log in and receive a JWT.